

# Lab 1 – Image Processing Basics

The purpose of this lab is to get familiar with the basic image processing functions in Python. In particular, you will learn how to load and display images using `scikit-image`, `matplotlib`, the `Python Imaging Library (PIL)` and `OpenCV`.

## Learning Objectives

- You know how to load images with various file formats (tif, jpg, gif, png, bmp, dicom) and store them as arrays.
- You know how to convert between different data types (`uint8` and `float`).
- You know how the data of color images are organized.
- You can select, process and reassemble 2D and 3D image regions.
- You know how to change the resolution of images and discuss the resulting effects.

## 1 Tasks

### 1.1 Grayscale Images

1. Make sure to have the relevant image processing Python-packages installed. Familiarize yourself with installing packages using `pip` or `conda`.
2. Load the image `lena_grayscale.gif`. Try different libraries: `scikit-image`, `Pillow`, and `OpenCV`. Print the return type and shape of the image array.
3. Determine the image size and the number of channels.
4. Display the image. Use at least two different methods (e.g., `matplotlib.pyplot.imshow(...)` and `cv2.imshow(...)`).
5. Save the image in different file formats such as `.tiff`, `.png` and `.bmp` file.

## 1.2 Color Images

1. Load the color image `lena_color.gif`.
2. Display the original image and all three color channels (R, G, B) separately.  
**Hint:** OpenCV uses **BGR order**, while PIL and scikit-image use **RGB**.
3. Convert the image to grayscale.

## 1.3 Medical Images: DICOM File Format

In medical imaging, the DICOM format is often used to store images. It includes not only pixel values but also metadata such as patient information, modality, and acquisition parameters.

1. Install the package `pydicom`.
2. Use the command `pydicom.dcmread(...)` to read the given DICOM file `brain_001.dcm`.
3. Display the corresponding image.
4. Print the metadata fields such as `PatientName`, `Modality`, `PixelSpacing`. Which additional metadata are contained in the file?
5. Flip the image both horizontally and vertically.
6. Convert the image to `uint8`. Display the resulting image `g`. Was the conversion correct? If not—what might be the issue?
7. This time, prior to again converting the image `f` to `uint8`, modify the image in such way that all pixel values are in the range `[0, 255]`. Then convert the original image and display the result.
8. Convert the image `g` from `uint8` to a floating point representation and display the result.

## 1.4 Image Sequences

The directory `brain` contains 20 sectional images (slices) that were obtained from a CT scan of a human head. The slices begin at the chin (slice No. 1), end at the forehead and are in parallel to the transversal planes (see Fig. 1). This results in a 3D representation of the head, with high spatial resolution in two dimensions and poor spatial resolution in vertical direction. For storing the images into a 3D array you can use the `np.stack(...)`. The result is a 3-D array, where the third index corresponds to the number of the image. Note: the patient's nose is pointing upwards.

1. Use a for-loop to display all individual images.
2. Save the image sequence as a `.gif` file (hint: use `Image.save` from the PIL library).

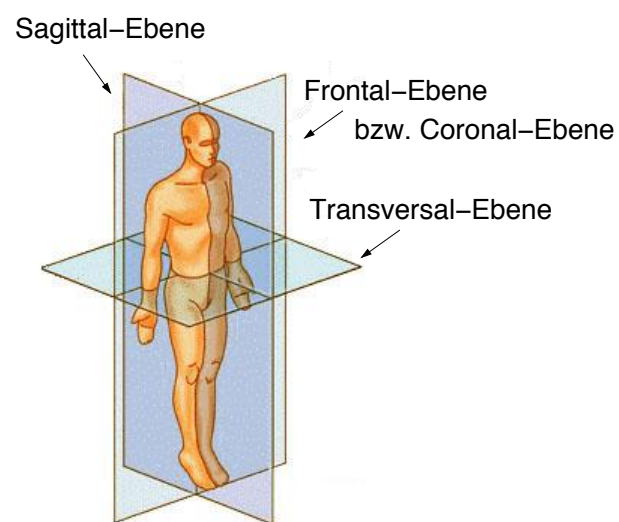


Figure 1: Different imaging planes.